MACHINE LEARNING

>:_ FACULTAD DE INGENIERÍA

Universidad Rafael Landívar
Identidad Jesuita en Guatemala

POSGRADOS LANDÍVAR

Laboratorio 2.4

Regresíon Lineal Múltiple

La data contiene las siguientes columnas:

- **'R.D.Spend': Gastos anuales de la empresa en investigación y desarrollo**
- **'Administration': Gastos anuales de la empresa en administración**
- **'Marketing Spend': Gastos anuales de la empresa en mercadeo**
- **'State': Estado en el que se encuentra la empresa**
- **'Profit': Ganancias anuales de la empresa**

Import Libraries

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

In [2]:

```
dataset = pd.read_csv('50_Startups.csv',header=0)
```

In [3]:

```
dataset.head()
```

Out[3]:

	R&D Spend	Administration	Marketing Spend	State	Profit
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

In [4]:

```
dataset.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 50 entries, 0 to 49
Data columns (total 5 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   R&D Spend        50 non-null     float64
 1   Administration   50 non-null     float64
 2   Marketing Spend  50 non-null     float64
 3   State            50 non-null     object
 4   Profit           50 non-null     float64
dtypes: float64(4), object(1)
memory usage: 2.1+ KB
```

In [5]:

```
dataset.describe()
```

Out[5]:

	R&D Spend	Administration	Marketing Spend	Profit
count	50.000000	50.000000	50.000000	50.000000
mean	73721.615600	121344.639600	211025.097800	112012.639200
std	45902.256482	28017.802755	122290.310726	40306.180338
min	0.000000	51283.140000	0.000000	14681.400000
25%	39936.370000	103730.875000	129300.132500	90138.902500
50%	73051.080000	122699.795000	212716.240000	107978.190000
75%	101602.800000	144842.180000	299469.085000	139765.977500
max	165349.200000	182645.560000	471784.100000	192261.830000

In [6]:

```
dataset = pd.get_dummies(dataset)
```

In [7]:

```
dataset.columns
```

Out[7]:

```
Index(['R&D Spend', 'Administration', 'Marketing Spend', 'Profit',
       'State_California', 'State_Florida', 'State_New York'],
      dtype='object')
```

In [8]:

```
dataset.head()
```

Out[8]:

	R&D Spend	Administration	Marketing Spend	Profit	State_California	State_Florida	State_New York
0	165349.20	136897.80	471784.10	192261.83	0	0	1
1	162597.70	151377.59	443898.53	191792.06	1	0	0
2	153441.51	101145.55	407934.54	191050.39	0	1	0
3	144372.41	118671.85	383199.62	182901.99	0	0	1
4	142107.34	91391.77	366168.42	166187.94	0	1	0

Exploración Visual

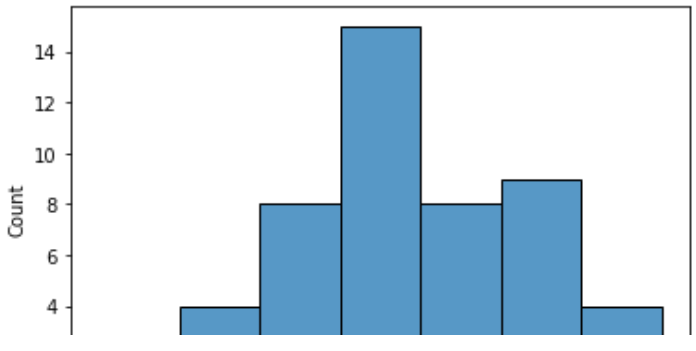
Siempre es importante entender la distribución de nuestra variable objetivo, ya que se asume que sigue una distribución normal

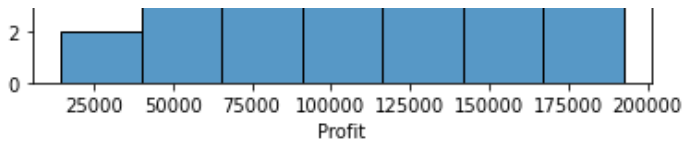
In [17]:

```
sns.histplot(dataset['Profit'])
```

Out[17]:

```
<AxesSubplot:xlabel='Profit', ylabel='Count'>
```





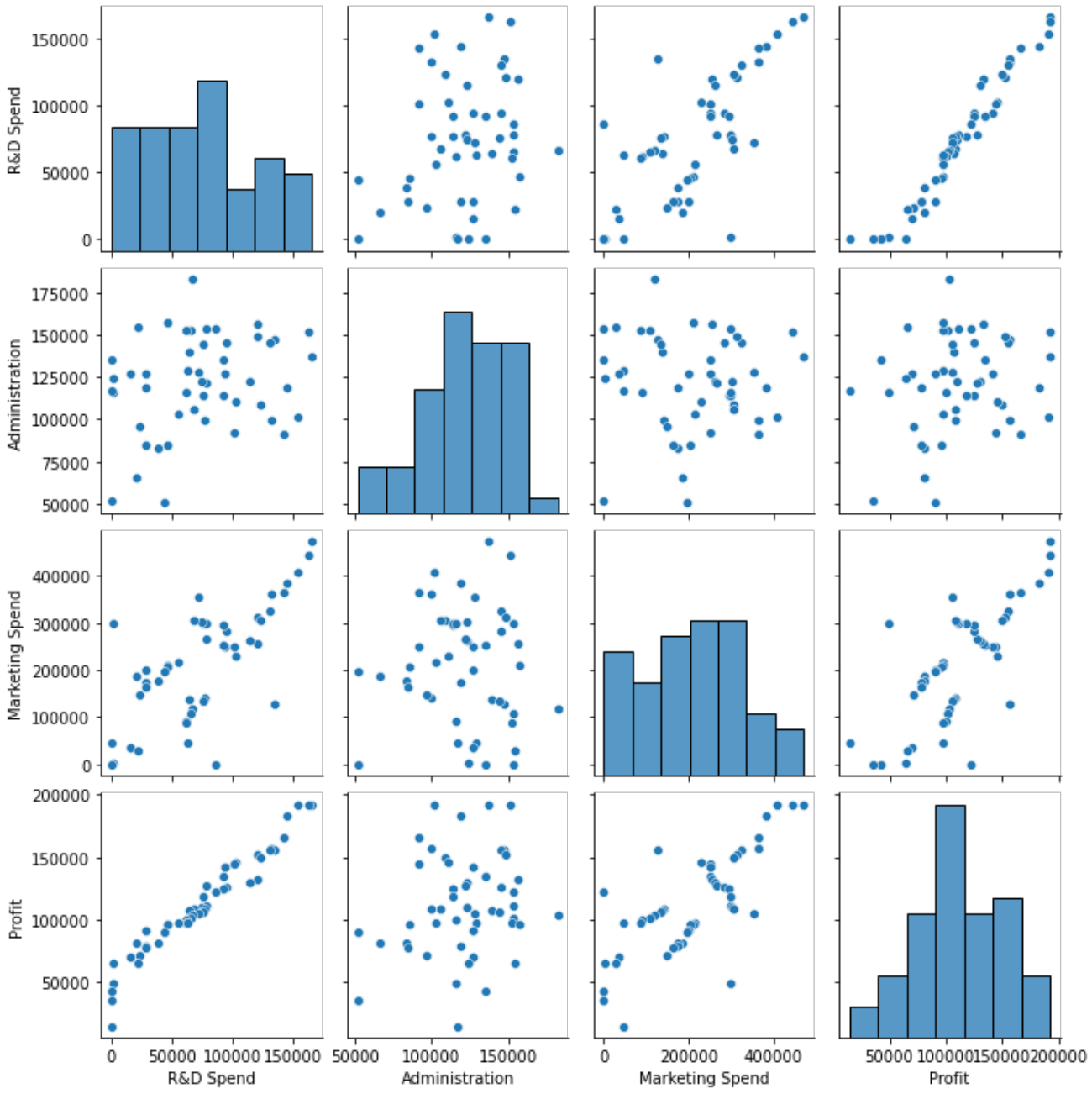
Una forma de entender las relaciones entre mis variables es graficar utilizando la función pairplot, la cual grafica en parejas de variables

In [18]:

```
sns.pairplot(dataset[['R&D Spend','Administration',"Marketing Spend","Profit"]])
```

Out[18]:

```
<seaborn.axisgrid.PairGrid at 0x7fa190662040>
```



Una buena práctica es seleccionar variables que no estén correlacionadas dentro de mi modelo. Para obtener la correlacción de manera gráfica podemos utilizar la función heatmap en combinación con el método corr.

In [19]:

```
sns.heatmap(dataset[['R&D Spend','Administration',"Marketing Spend","Profit"]].corr())
```

Out[19]:

```
<AxesSubplot:>
```

Entrenamiento de Modelo de Regression Lineal

In [20]:

```
X = dataset[['R&D Spend','Administration','Marketing Spend','State_California','State_Fl
orida']]
y = dataset['Profit']
```

In [ ]:

```

```

Train Test Split

In [21]:

```
from sklearn.model_selection import train_test_split
```

In [22]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=99
)
```

Creando y Entrenando el Modelo

In [26]:

```
from sklearn.linear_model import LinearRegression
```

In [27]:

```
lm = LinearRegression()
```

In [28]:

```
lm.fit(X_train,y_train)
```

Out[28]:

```
LinearRegression()
```

Evaluación del Modelo

Evaluemos el modelo e interpretemos sus coefficientes

In [29]:

```
#Intercepto
print(lm.intercept_)
```

```
47169.90797768808
```

In [129]:

```
#Coeficientes
coeff_df = pd.DataFrame(lm.coef_,X.columns,columns=['Coefficient'])
coeff_df
```

Out[129]:

	Coefficient
R&D Spend	0.820135
Administration	-0.009123
Marketing Spend	0.023009
State_California	-1352.011873
State_Florida	-74.732930

Interpretación de los coeficientes:

- **Manteniendo todas las demás variables constantes, por cada dólar que se gaste en R&D Spend, podemos esperar un aumento en el profit de $0.82**
- **Manteniendo todas las demás variables constantes, por cada dólar que se gaste en Administration, podemos esperar una reducción en el profit de $0.009**
- **Manteniendo todas las demás variables constantes, por cada dólar que se gaste en Marketing Spend, podemos esperar un aumento en el profit de $0.02**
- **Manteniendo todas las demás variables constantes, estar en el estado de California, en comparación con el estado de Nueva York, hace que el profit se reduzca en $1,352.**
- **Manteniendo todas las demás variables constantes, estar en el estado de Florida, en comparación con el estado de Nueva York, hace que el profit se reduzca en $74.**

Visualizar la Efficiencia de un Modelo

Una forma sencilla de evaluar un modelo de regresión es comparando los valores reales contra los valores predichos en un scatter plot. La línea en el gráfico sirve de referencia para indicar el comportamiento de un modelo con predicciones perfectas.

In [30]:

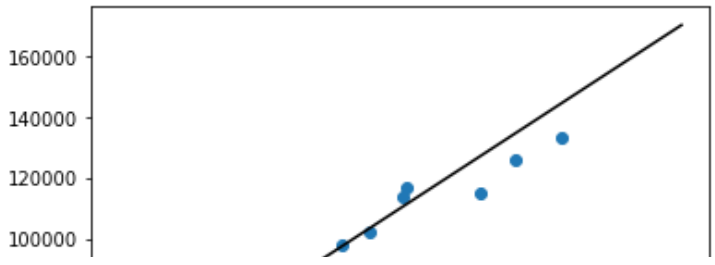
```
predictions = lm.predict(X_test)
```

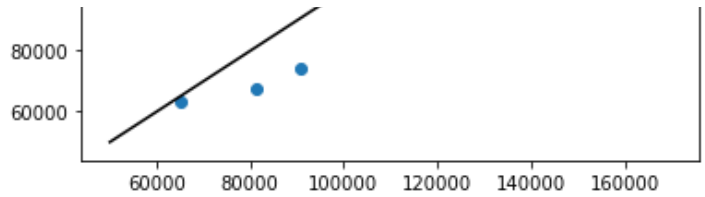
In [31]:

```
plt.scatter(y_test,predictions)
plt.plot([50000, 170000], [50000, 170000], color = 'black')
```

Out[31]:

```
[<matplotlib.lines.Line2D at 0x7fa168044f40>]
```



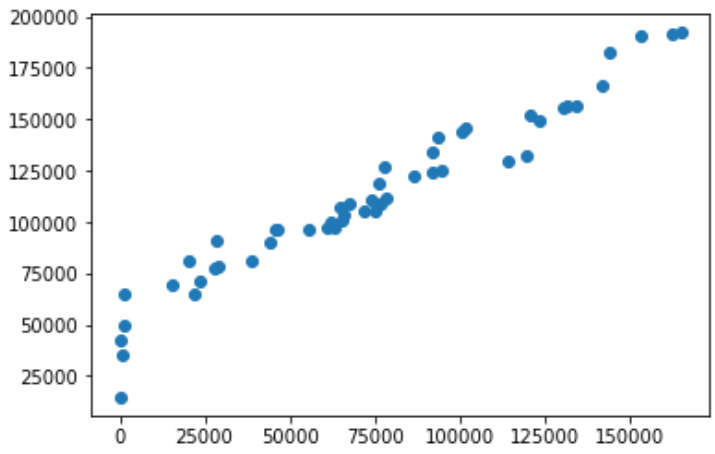


In [37]:

```
plt.scatter(dataset['R&D Spend'],dataset['Profit'])
```

Out[37]:

```
<matplotlib.collections.PathCollection at 0x7fa190d4fb50>
```



Obteniendo el coeficiente de correlación entre mis predicciones y el y-real puedo entender que tan efectivo es mi modelo

In [52]:

```
np.corrcoef(y_test,predictions)[0][1]
```

Out[52]:

```
0.9501983359183573
```

Residual Histogram

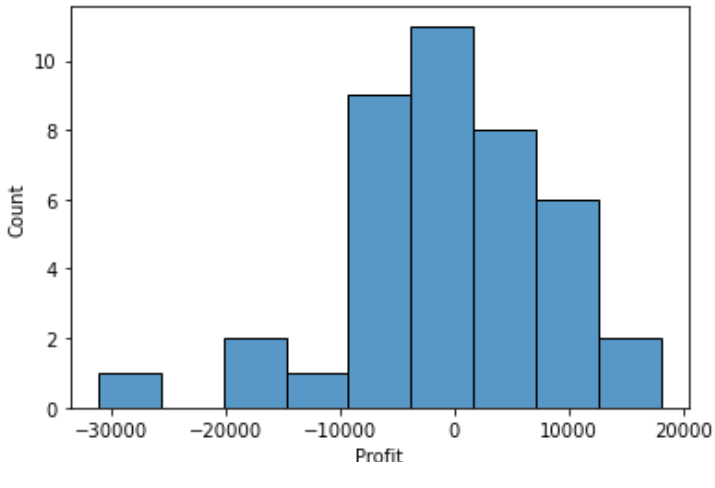
Recordemos que dos de los supuestos de una regresión lineal es que el error está centrado en cero y sigue una distribución normal. Por ende, es una buena práctica graficar los residuales.

In [46]:

```
predictions_train = lm.predict(X_train)
```

In [47]:

```
sns.histplot((y_train-predictions_train),bins=9);
```



Regression Evaluation Metrics

Aquí hay tres métricas de evaluación comunes para problemas de regresión:

Mean Absolute Error (MAE) es la media del valor absoluto de los errores:

$$\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

Mean Squared Error (MSE) es la media de los errores al cuadrado:

$$\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Root Mean Squared Error (RMSE) es la raíz cuadrada de la media de los errores al cuadrado:

$$\sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

Comparando estas métricas:

- **MAE** es el más fácil de entender, porque es el error medio.
- **MSE** es más popular que MAE, porque MSE "castiga" errores mayores, lo que tiende a ser útil en el mundo real.
- **RMSE** es incluso más popular que MSE, porque RMSE se puede interpretar en las unidades "y".

Todas estas son funciones de pérdida, porque queremos minimizarlas.

In [48]:

```
from sklearn import metrics
```

In [49]:

```
print('MAE:', metrics.mean_absolute_error(y_test, predictions))
print('MSE:', metrics.mean_squared_error(y_test, predictions))
print('RMSE:', np.sqrt(metrics.mean_squared_error(y_test, predictions)))
```

```
MAE: 7421.921617995084
MSE: 85616476.34474911
RMSE: 9252.917180259916
```

Significancia de los coeficientes

In [50]:

```
import statsmodels.api as sm
```

In [51]:

```
X2 = sm.add_constant(X_train)
est = sm.OLS(y_train, X2)
est2 = est.fit()
print(est2.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                 Profit   R-squared:                       0.957
Model:                            OLS   Adj. R-squared:                  0.951
Method:                 Least Squares   F-statistic:                     151.4
Date:                Fri, 03 Mar 2023   Prob (F-statistic):           3.23e-22
Time:                        08:24:57   Log-Likelihood:                -420.56
No. Observations:                  40   AIC:                             853.1
Df Residuals:                      34   BIC:                             863.3
Df Model:                           5
Covariance Type:            nonrobust
```

```
==============================================================================
                        coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const              4.717e+04   8093.202      5.828      0.000    3.07e+04    6.36e+04
R&D Spend             0.8201      0.053     15.599      0.000       0.713       0.927
Administration       -0.0091      0.063     -0.144      0.887      -0.138       0.120
Marketing Spend       0.0230      0.019      1.191      0.242      -0.016       0.062
State_California -1352.0119   3810.480     -0.355      0.725   -9095.839    6391.815
State_Florida      -74.7329   3764.195     -0.020      0.984   -7724.498    7575.032
==============================================================================
Omnibus:                       12.497   Durbin-Watson:                   1.782
Prob(Omnibus):                  0.002   Jarque-Bera (JB):               14.566
Skew:                          -0.982   Prob(JB):                     0.000687
Kurtosis:                       5.210   Cond. No.                     1.53e+06
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specifie
d.
[2] The condition number is large, 1.53e+06. This might indicate that there are
strong multicollinearity or other numerical problems.
```

[redacted]

En el resumen anterior, se puede observar que las dos únicas variables significativas son el intercepto y R&D Spend, por lo que entrenaremos un nuevo modelo solamente con esas dos variables y compararemos resultados.

In [53]:

```
lm2 = LinearRegression()
lm2.fit(X_train[['R&D Spend']],y_train)
```

Out[53]:

```
LinearRegression()
```

In [54]:

```
predictions2 = lm2.predict(X_test[['R&D Spend']])

np.corrcoef(y_test,predictions2)[0][1]
```

Out[54]:

```
0.9381960295948544
```

In [55]:

```
#Correlacion del modelo anterior
np.corrcoef(y_test,predictions)[0][1]
```

Out[55]:

```
0.9501983359183573
```

In [57]:

```
lm2.coef_
```

Out[57]:

```
array([0.8657168])
```